

A Graph-Based Control Interface for Traffic Signals on Heterogeneous Road Networks

Bertil Braun

contact@bertil-braun.de

Abstract

We present a traffic-signal control interface in which a shared graph neural network assigns scores to individual traffic movements. Each junction converts these scores into its own variable-sized set of legal signal phases using a deterministic incidence matrix. Directed corridor nodes provide traffic context, while movement nodes represent controlled input-to-output paths through junctions. Typed mean aggregation produces one scalar per movement; phase definitions and signal timing remain outside the learned network. This makes graph size and junction-specific action count independent of the learned parameter shapes. PPO experiments evaluate the interface on unseen synthetic grid geometries, altered signal coverage, and five heterogeneous city graphs. The policies retained performance across unseen geometries within the synthetic grid family, while changes in signal coverage exposed sensitivity to a signal-coverage distribution shift. A single trained city-policy instance executed across all five city graphs, with heterogeneous outcomes. These results provide feasibility evidence rather than a general estimate of transfer to arbitrary road networks.

1 Introduction

Traffic-signal action spaces are local. A three-arm junction, a regular four-arm junction, and a junction with protected turns need not have the same number or meaning of phases. Consequently, a fixed output called “phase 2” does not have reusable semantics across road networks. Padding such a head changes its tensor shape but does not give the padded phase indices shared meaning across junctions.

We present a traffic-signal control interface in which a shared graph neural network assigns scores to individual traffic movements. Each junction converts these scores into its own variable-sized set of legal signal phases using a deterministic incidence matrix. The central design choice is therefore a narrow boundary: learning prioritizes comparable movement objects, while deterministic code constructs and operates each junction’s valid local choices.

The resulting system combines a reusable movement-level graph representation, deterministic construction of junction-specific action spaces, and a feasibility evaluation across synthetic and city simulations. Its variable graph and action dimensions are a structural property of the design. We distinguish a structural property established by construction, implementation validation over heterogeneous graphs, and bounded empirical evidence about learned performance. Accordingly, variable graph and action dimensions are the primary result, trainability and cross-scenario execution are secondary validation, and learned–baseline outcomes are exploratory comparisons under unmatched controller timings. The empirical evaluation is organized around three research questions:

RQ1 — Transfer within the synthetic generator family.

How does the trained policy behave on unseen grid sizes and aspect ratios produced by the same generator?

RQ2 — Distribution shift.

What happens when signal coverage changes?

RQ3 — City feasibility.

Can the same trained policy execute on several heterogeneous city simulations, and how variable are the observed outcomes?

Movement pressure and movement-structured control have substantial prior lineage. Max-pressure methods score compatible stages from constituent flows [1], while PressLight connects pressure to learned control [2]. FRAP learns phase demand and pairwise phase competition from movement features, explicitly structuring the learned phase values around conflict and symmetry [3]. TransferLight is the closest architectural comparison: its learned directed hierarchy aggregates lane segments into movement representations, movements into phase representations, and interactions among phases before producing phase energies [4]. It also uses weight-tied decentralized agents, domain-randomized training networks, and learned movement-to-phase semantics. In contrast, our learned actor operates on a city-level LaneGroup/Movement context graph and stops at one scalar per movement; phase membership, phase enumeration, incidence summation, and timing remain deterministic. The narrower contribution is this transparent boundary between shared movement scoring and automatically constructed local action spaces, rather than movement-based control itself or a broader transfer claim.

Scope. This report evaluates an implementation and architectural interface rather than proposing a new reinforcement-learning algorithm. The experiments evaluate transfer within the specified simulation families and execution across heterogeneous action spaces. They do not establish general transfer across arbitrary road networks.

2 Movement-level control interface

2.1 Intuition and control objects

A *movement* is one legal controlled path from an incoming road corridor to an outgoing corridor; straight travel is a movement as well as a turn. A *phase* is a compatible set of movements that may receive green together. The controller chooses one phase per junction, rather than setting individual lamps independently.

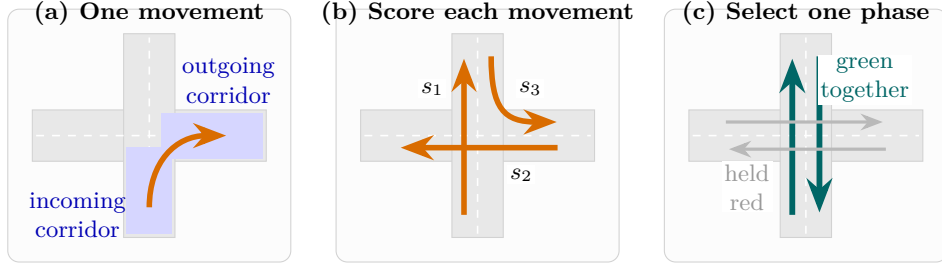


Figure 1: The control vocabulary. The shared model scores movements individually; each junction supplies its own compatible phase sets. The phase shown is illustrative, not a universal template.

The implementation groups consecutive directed road segments into a **LaneGroup** when an unsignalized continuation is unambiguous. Opposite directions remain separate because their queues, speeds, and destinations differ. At a controlled junction, a legal input-to-output connection becomes a movement node. The junction owns its movements and phases but is not itself a GNN node.

Table 1: Roles in the representation and action interface.

Object	Represents	Used for
lane group	One direction of a road corridor	Queue, speed, occupancy, capacity, arrivals, departures, and downstream space
movement	One controlled input-to-output path	Turn-local demand, controlled-link count, and recent green state
phase	One compatible local movement set	A junction-specific selectable action

At each decision, features are refreshed from SUMO, the shared GNN emits one score per movement, and a junction-local incidence matrix converts those scores to phase logits. An availability mask enforces minimum green; the runtime inserts yellow when an accepted target changes. Only movement scoring is learned.

2.2 Typed graph and message-passing architecture

Each movement m has one input LaneGroup $i(m)$ and one output LaneGroup $o(m)$, giving four directed information relations:

$$L_{\text{in}} \rightarrow M, \quad L_{\text{out}} \rightarrow M, \quad M \rightarrow L_{\text{in}}, \quad M \rightarrow L_{\text{out}}.$$

The output-to-movement relation returns downstream storage context to a movement that would feed that corridor. Unsignalized pass-through connections are weighted $L \rightarrow L$ edges with $w_{ql} = \exp(-t_{\text{ff}}/30\text{s})$.

Let ρ denote ReLU, $\parallel$ concatenation, and x_l, x_m the LaneGroup and movement feature vectors. The implementation first creates

$$h_l^{(0)} = \rho(E_L x_l), \tag{1}$$

$$h_m^{(0)} = \rho\left(E_M[x_m \parallel h_{i(m)}^{(0)} \parallel h_{o(m)}^{(0)}]\right). \tag{2}$$

For relation r , target v , and source embeddings z_q , define the transformed mean

$$\mathcal{A}_r^{(k)}(v; z) = \frac{1}{\max(1, |\mathcal{N}_r(v)|)} \sum_{q \in \mathcal{N}_r(v)} w_{qv} W_r^{(k)} z_q,$$

where $w_{qv} = 1$ except on unsignalized connector edges. The incoming and outgoing messages to movement m are

$$a_{\text{in},m}^{(k)} = \mathcal{A}_{L_{\text{in}} \rightarrow M}^{(k)}(m; h_L^{(k)}), \quad a_{\text{out},m}^{(k)} = \mathcal{A}_{L_{\text{out}} \rightarrow M}^{(k)}(m; h_L^{(k)}). \tag{3}$$

One block first updates the movement embedding:

$$h_m^{(k+1)} = \rho\left(U_M^{(k)}[h_m^{(k)} \parallel a_{\text{in},m}^{(k)} \parallel a_{\text{out},m}^{(k)}]\right). \tag{4}$$

It then forms the return messages

$$b_{\text{in},l}^{(k)} = \mathcal{A}_{M \rightarrow L_{\text{in}}}^{(k)}(l; h_m^{(k+1)}), \quad b_{\text{out},l}^{(k)} = \mathcal{A}_{M \rightarrow L_{\text{out}}}^{(k)}(l; h_m^{(k+1)}), \tag{5}$$

$$c_l^{(k)} = \mathcal{A}_{L \rightarrow L}^{(k)}(l; h_L^{(k)}), \tag{6}$$

and updates the LaneGroup embedding:

$$h_l^{(k+1)} = \rho\left(U_L^{(k)}[h_l^{(k)} \parallel (b_{\text{in},l}^{(k)} + c_l^{(k)}) \parallel b_{\text{out},l}^{(k)}]\right). \tag{7}$$

Every relation has its own linear map, aggregation is a mean rather than attention, and return messages use the just-updated embeddings from (4). An empty neighbourhood produces the zero vector. After two blocks, an MLP maps every $h_m^{(2)}$ to

one scalar s_m . Each junction is treated as a parameter-sharing local agent: the critic mean-pools that junction’s movement embeddings, returns one value, and its local reward and value sequence produce a separate temporal GAE stream. For PPO updates, variable-size state graphs are concatenated as disconnected packed graphs; junctions with matching local movement and phase dimensions are grouped for batched value and incidence operations, and minibatches are budgeted by junction/action samples rather than padded to a universal graph or action size.

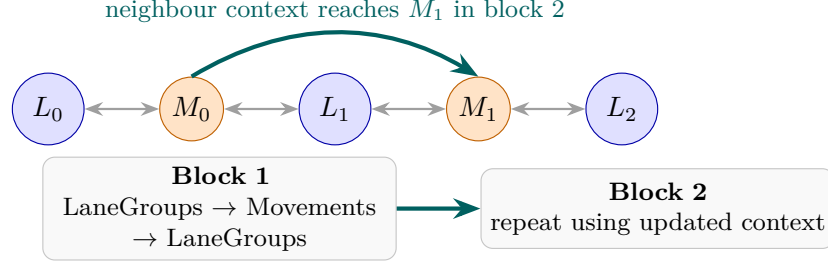


Figure 2: The typed representation and update order. With this order, two blocks allow information from one movement to reach another through a shared LaneGroup.

The parameter matrices above are shared over all nodes and edges of a type. Their shapes depend only on feature and hidden dimensions. The parameter shapes are therefore independent of graph and action-space size.

2.3 Deterministic local action spaces

The build pipeline obtains controlled links and request-conflict data from SUMO `netconvert` [5]. Links constrained to activate together form atomic groups; groups with internal SUMO conflicts are rejected. Two groups are incompatible when SUMO reports a conflict in either direction or different incoming approaches merge into the same outgoing edge. Bron–Kerbosch enumeration [6] produces all maximal compatible group sets, each of which becomes a selectable phase. Here, maximal means that no additional compatible group can be added, not that the phase has maximum size.

For junction j , $A_j \in \{0, 1\}^{|P_j| \times |M_j|}$ records whether phase p enables movement m . The local logits are

$$\ell_j = A_j \mathbf{s}_j, \quad \ell_{j,p} = \sum_{m \in M_j} A_{j,p,m} s_m.$$

For example,

$$A_j = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}, \quad \mathbf{s}_j = [1.2 \quad 0.7 \quad 0.6 \quad -0.4]^\top, \quad A_j \mathbf{s}_j = [1.9 \quad 0.2]^\top.$$

The resulting logits favor the first phase; masking and categorical sampling follow before the runtime applies any signal transition. A different junction may provide a 6×11 matrix without changing the scorer.

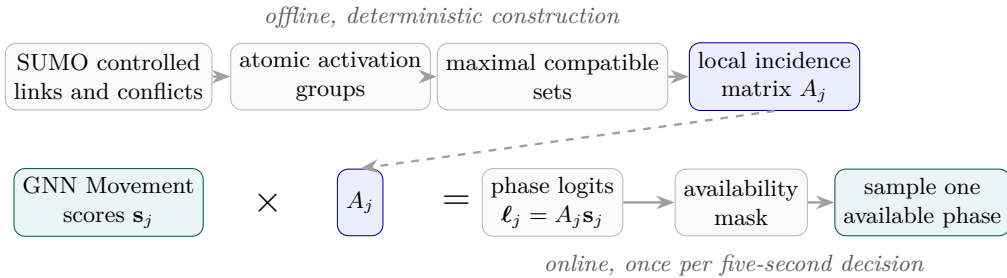


Figure 3: Offline phase construction and online action selection. The fixed incidence matrix changes with each junction; the learned scorer does not.

Sum aggregation intentionally restricts the policy class: a phase logit is the additive utility of its enabled movements, so the actor cannot directly represent arbitrary within-phase interactions. Phases sharing movements also have correlated logits, and when scores are positive a larger phase can receive a higher logit simply because it has more terms. The restriction keeps the learned/local interface transparent, but neither its phase-size effect nor alternatives such as normalized or learned phase aggregation were evaluated separately.

3 Experimental design and reproducibility

3.1 Training protocol and local reward

PPO [7] optimizes the complete policy. All reported studies use two message-passing blocks, 5 s decisions, immediate 3 s yellow on a change, one decision of minimum green, four PPO epochs, 200 decisions per rollout, and entropy coefficient 0.001. Rollout counts are allocated to approximately balance junction/action samples across differently sized training graphs.

The code assigns one reward to each junction j per decision interval $\Delta = 5$ s. Qualitatively, the local reward encourages vehicles to move through and leave the incoming approaches while penalizing deceleration and queues. Let I_j be its unique

incoming lanes, $D_j = \sum_{l \in I_j} D_l$ their total length, n_l the vehicle count, $\bar{v}_l$ mean speed, $v_l^{\max}$ speed limit, and $V_j(t)$ the set of vehicle IDs on those lanes. With $f_l(t) = \text{clip}(\bar{v}_l(t)/v_l^{\max}, 0, 1)$, the implemented terms are:

Table 2: Exact local reward terms for the reported runs. Values are sampled at decision boundaries.

Term	Definition and interpretation	Unit
Progress p_j	$D_j^{-1} \sum_{l \in I_j} n_l(t) f_l(t)$: speed-normalized moving-vehicle density	veh/m
Discharge d_j	$ V_j(t - \Delta) \setminus V_j(t) / (\Delta D_j)$: IDs leaving the incoming-lane set	veh/(m s)
Braking b_j	$(\Delta D_j)^{-1} \sum_{l \in I_j} n_l(t) [\bar{v}_l(t - \Delta) - \bar{v}_l(t)]_+ / v_l^{\max}$	veh/(m s)
Gridlock q_j	$D_j^{-1} \sum_{l \in I_j} n_l(t) (1 - f_l(t))$: speed-deficit density	veh/m

The first braking sample is zero because no preceding snapshot exists. The per-junction reward is

$$r_j = \text{clip}_{[-1,1]}(p_j + 10d_j - 10b_j - 0.02q_j).$$

Global delay, completed-network flow, direct throughput, phase switching, and teleport terms have zero weight in these runs. Because the terms have different units and are manually weighted, the final reward is a dimensionless optimization objective. It is a local surrogate, not a guarantee of improvement in network-level throughput, completion, or wait density: for example, discharge from one junction can move vehicles into a congested downstream region.

3.2 Evaluation metrics

Every episode records per-seed metrics before arithmetic means are formed. For an episode of T simulated seconds, let C be trips with a SUMO `tripinfo` arrival after warm-up and D the initial post-warm-up population plus subsequent departures.

Table 3: Primary evaluation metrics and aggregation.

Metric	Implemented definition	Unit
Throughput	$C/(T/3600)$	veh/h
Completion	C/D ; vehicles still active at the horizon remain incomplete	fraction or %
Wait density	Time mean of $\sum_l W_l(t) / \sum_l D_l$, where W_l is SUMO’s total waiting time of vehicles currently on lane l	s/m

Wait density uses all non-internal network lanes at every simulated second and exposes queues left at the horizon. Grid results separate three policy-training seeds from six held-out traffic seeds.

3.3 Baseline protocol and study matrix

All policies receive the same saved SUMO network, routes, initial-occupancy range, warm-up, episode seed, conflict-derived phase set, minimum-green rule, and yellow transition. The learned and uniform-random policies choose every 5 s; fixed time cycles through phase order every 10 s; max pressure and queue recompute a target every 10 s. This mismatch is a timing confound: the comparisons evaluate the recorded controller implementations and do not isolate learning or policy architecture from control frequency. Max pressure observes the instantaneous halting counts in the downstream detector regions of the movement’s input and output LaneGroups and scores $n_{\text{in}}^{\text{halt}} - n_{\text{out}}^{\text{halt}}$; queue observes the input detector count only. Both sum movement scores through the same A_j and retain the current phase on a tie. Fixed time and uniform random use no traffic observation. The 10 s durations and all other baseline parameters were fixed in the experiment configurations; the records contain no baseline-tuning study. Max pressure is the reference for RQ1 and RQ2. For RQ3, city-specific comparisons also report the highest-throughput non-learned baseline: max pressure in Karlsruhe, queue in Mannheim, and fixed time in Stuttgart, Heidelberg, and Freiburg.

- **RQ1:** three mixed-grid training seeds, trained on five square or rectangular geometries whose longest axis is five; final evaluation on ten geometries at demands 0.6, 0.7, 0.8, including 6×6 , with six held-out traffic seeds.
- **RQ2:** a full-coverage-trained 6×6 policy evaluated after removing signals, plus one separate training seed exposed to five 50%-coverage 4×4 layouts and evaluated on held-out 25%–100% layouts.
- **RQ3:** one city policy-training seed with rollouts from Karlsruhe, Mannheim, Heidelberg, and Freiburg. Stuttgart contributed no training rollouts. Although periodic Stuttgart evaluations were available, the reported iteration-60 checkpoint was the fixed terminal checkpoint rather than one selected using Stuttgart performance. It was evaluated for 1200 s at demand scale 1.0 on three traffic seeds in all five cities.

All reported comparisons use libsumo/sumolib 1.27.1 as resolved by `uv.lock`.

Sampled execution is primary because it corresponds to the stochastic policy optimized during PPO training. Greedy execution performed worse in the city evaluation. The sampled results therefore do not imply equivalent performance under deterministic argmax execution. Deployment under deterministic action selection would require separate validation.

4 Structural property and empirical results

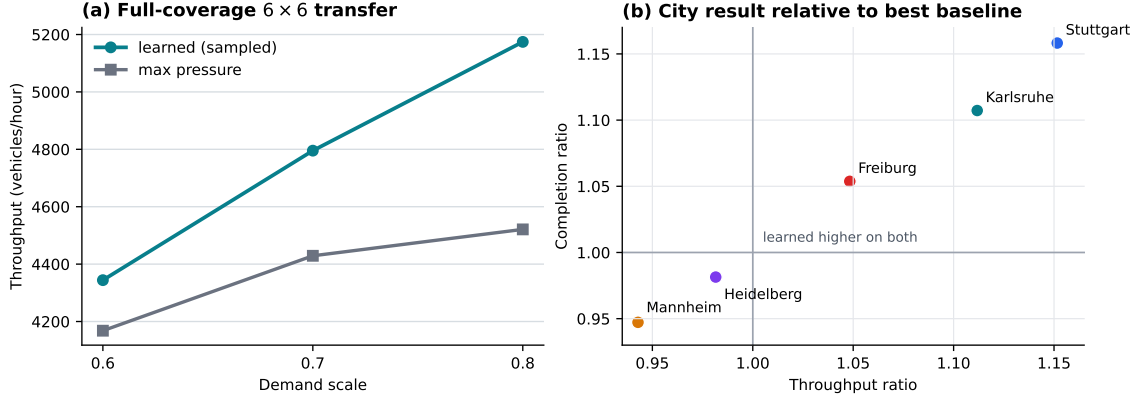


Figure 4: Observed feasibility results. Left: sampled 6×6 performance by demand. Right: iteration-60 city outcomes; each city is normalized by its own highest-throughput non-learned baseline, so baseline identities differ across cities.

Structural property — variable graph and action dimensions. Equations (4)–(7) use shared parameter shapes and return one scalar for every movement presented to them. Every junction may independently provide A_j with its own numbers of rows and columns; phase construction is outside the GNN. This establishes variable graph and action dimensions by construction. Execution on graphs ranging from 41 to 84 controlled junctions and phase-count ranges of 2–12 validates that the implementation supports the stated structural range.

All learned–baseline comparisons below inherit the decision-interval confound described in Section 3: they compare the recorded implementations, not learning at a matched control frequency.

RQ1 — transfer within the synthetic generator family. On the unseen-size 6×6 condition, sampled control had higher throughput and completion than max pressure at all three demands; completion differences were 3.5, 6.6, 10.6 percentage points. Full means and separate trained-policy and traffic-seed dispersion summaries are reported in Appendix B. Orientation-swapped rectangular grids produced closely matched throughput and completion. These findings are consistent with size and aspect-ratio reuse within the matched generator.

RQ2 — distribution shift. The full-coverage-trained policy’s throughput and completion deteriorated relative to max pressure at 50% and 25% signal coverage, despite the graph remaining structurally executable. In the separate 4×4 study, sampled throughput differences from max pressure ranged from +5.7 to −108.0 vehicles/hour across coverage conditions, with uncertainty especially wide at 100%; Appendix B gives the paired intervals. The result distinguishes architectural compatibility from robustness and is consistent with sensitivity to the controller-distribution shift.

RQ3 — city feasibility. The city evaluation reports one policy-training run rather than an estimate across independent training runs. The policy executed on Karlsruhe, Mannheim, Heidelberg, and Freiburg, all represented in rollout training, and on rollout-held-out Stuttgart. Stuttgart therefore provides the only city-level test of generalization to a network absent from rollout training in this experiment; the other four results primarily demonstrate shared-parameter execution across heterogeneous training domains. Outcomes varied: sampled control had higher throughput and completion than every baseline in Karlsruhe and Stuttgart, trailed queue in Mannheim, was close to fixed time in Heidelberg, and traded higher throughput and completion for higher wait density relative to fixed time in Freiburg. Figure 4b uses the strongest non-learned throughput baseline separately for each city.

5 Limitations and conclusion

Limitations. The experiments evaluate the complete implementation and do not isolate individual architectural choices. They therefore do not determine the separate contributions of typed relations, two message-passing blocks, or sum-based movement-to-phase aggregation; the latter also introduces an unevaluated phase-size bias. Because only maximal compatible movement sets are enumerated, a compatible strict subset is unavailable whenever another compatible atomic group can be added. This is a deliberate restriction of the selectable policy space and may prevent a controller from withholding green from an otherwise compatible movement. Phase enumeration was inexpensive for the evaluated junctions, which produced at most 12 phases, but maximal-compatible-set enumeration can scale poorly for substantially more complex conflict graphs.

The evaluated networks belong to bounded synthetic and city simulation families rather than arbitrary topology. The coverage study shows sensitivity to the controller distribution, and the city study’s one policy-training seed and three traffic seeds per city do not estimate variation across independent training runs.

The generated phase sets inherit the assumptions and fidelity of the SUMO network and implemented conflict rules; they are action sets accepted by the implemented construction, not formally verified real-world signal plans. Sampled execution outperformed greedy execution in the city study, so the stochastic results do not establish an equally effective deterministic controller.

Conclusion. The implemented interface separates reusable learned scoring from local signal structure: a shared typed GNN emits one scalar per movement, while deterministic incidence matrices translate those scores into heterogeneous phase spaces. The experiments add bounded feasibility evidence within synthetic and city simulation families, while the coverage and greedy-execution results show that executable structure alone does not ensure empirical robustness.

References

- [1] Pravin Varaiya. Max pressure control of a network of signalized intersections. *Transportation Research Part C: Emerging Technologies*, 36:177–195, 2013.
- [2] Hua Wei, Chacha Chen, Guanjie Zheng, Kan Wu, Vikash Gayah, Kai Xu, and Zhenhui Li. PressLight: Learning max pressure control to coordinate traffic signals in arterial network. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1290–1298, 2019.
- [3] Guanjie Zheng, Yuanhao Xiong, Xinshi Zang, Jie Feng, Hua Wei, Huichu Zhang, Yong Li, Kai Xu, and Zhenhui Li. Learning phase competition for traffic signal control. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, pages 1963–1972, 2019.
- [4] Johann Schmidt, Frank Dreyer, Sayed Abid Hashimi, and Sebastian Stober. TransferLight: Zero-shot traffic signal control on any road-network. *arXiv preprint arXiv:2412.09719*, 2024.
- [5] Pablo Alvarez Lopez, Michael Behrisch, Laura Bieker-Walz, Jakob Erdmann, Yun-Pang Flötteröd, Robert Hilbrich, Leonhard Lücken, Johannes Rummel, Peter Wagner, and Evamarie Wießner. Microscopic traffic simulation using SUMO. In *2018 21st International Conference on Intelligent Transportation Systems*, pages 2575–2582, 2018.
- [6] Coen Bron and Joep Kerbosch. Algorithm 457: Finding all cliques of an undirected graph. *Communications of the ACM*, 16(9):575–577, 1973.
- [7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

A Implementation and scenario details

Artifact statement. Code, committed scenarios, and instructions are available at [repository snapshot ea47985](#); `uv.lock` resolves `libsumo` and `sumolib` 1.27.1. The reported protocols use `configs/training/grid_shape_generalization_mixed_2hop_gate_30.yaml`, plus the following files under `configs/training/`:

- `grid_signal_coverage_6x6_evaluation.yaml`;
- `grid_coverage_generalization_4x4_train50_2hop_30.yaml` and `grid_coverage_generalization_4x4_evaluation.yaml`;
- `city_stuttgart_visible_validation_local_reward_2hop_60.yaml`.

Evaluation seeds are recorded in those configurations and the linked result documentation. `scripts/analyze_grid_generalization_results.py` regenerates grid summaries, and `scripts/plot_technical_report_results.py` regenerates Figure 4 from saved CSVs. The reported trained checkpoints and raw evaluation summaries are preserved in a separate experiment archive, are not part of the Git repository, and are available from the author.

Table 4: Anchored training and control settings.

Setting	Value
Decision interval / yellow	5 s / immediate 3 s
Minimum green	one decision
Message-passing blocks / hidden dimension	2 / 64
Decisions per rollout / PPO epochs	200 / 4
Entropy coefficient / reward clip	0.001 / $[-1, 1]$
Progress/discharge/braking/gridlock	1/10/10/0.02
Global/flow/throughput/switch/teleport weight	0
Initial occupancy / warm-up	6–8% / 15 s

Table 5: Structural diversity of the city scenarios. Phase range is the minimum–maximum number of selectable phases per policy-controlled junction.

City	Controllers	LaneGroups	Movements	Phase Range
Karlsruhe	41	286	271	2–10
Mannheim	84	708	467	2–7
Stuttgart	69	519	508	2–12
Heidelberg	56	408	365	2–10
Freiburg	58	425	416	2–11

B Detailed evaluation

Table 6: Full-coverage 6×6 means and distinct SD sources. Learned means average three trained-policy means, each formed over six traffic seeds; their SD is between trained policies. Max-pressure mean and SD are across six traffic seeds. The SD columns are not directly comparable stability estimates.

Demand	Metric	Learned mean	Training-seed SD	MP mean	Traffic-seed SD
0.6	Throughput/h	4344	13	4168	126
	Completion	87.2%	0.3 pp	83.6%	1.3 pp
	Wait density	0.077	0.008	0.082	0.062
0.7	Throughput/h	4795	9	4429	157
	Completion	84.8%	0.1 pp	78.2%	1.8 pp
	Wait density	0.114	0.009	0.119	0.120
0.8	Throughput/h	5174	23	4521	164
	Completion	82.2%	0.5 pp	71.5%	2.5 pp
	Wait density	0.175	0.018	0.131	0.022

Within each trained policy, traffic-seed SDs ranged from 121 to 168 vehicles/hour for throughput, 0.4 to 2.0 percentage points for completion, and 0.007 to 0.024 s/m for wait density across the three demands. Thus the small between-policy SDs in Table 6 should not be read as episode-level variability.

Table 7: Iteration-60 city means $\pm$ traffic-seed SD over three seeds. The baseline has the highest mean throughput among non-learned policies in that city. Wait density is in seconds per metre; these SDs do not include trained-policy variability.

City	Policy	Throughput/h	Completion	Wait density
Karlsruhe	learned sampled	3452 ± 314	$93.86 \pm 0.34\%$	0.075 ± 0.034
	max pressure	3105 ± 241	$84.77 \pm 8.12\%$	0.243 ± 0.226
Mannheim	learned sampled	3200 ± 563	$53.78 \pm 13.46\%$	0.796 ± 0.418
	queue	3394 ± 562	$56.78 \pm 11.96\%$	1.019 ± 0.563
Stuttgart	learned sampled	4216 ± 528	$50.93 \pm 9.13\%$	0.907 ± 0.597
	fixed time	3661 ± 154	$43.97 \pm 1.18\%$	0.741 ± 0.227
Heidelberg	learned sampled	3190 ± 127	$77.69 \pm 1.53\%$	0.181 ± 0.055
	fixed time	3250 ± 134	$79.16 \pm 2.19\%$	0.127 ± 0.044
Freiburg	learned sampled	2670 ± 225	$48.39 \pm 6.77\%$	1.108 ± 0.449
	fixed time	2547 ± 35	$45.92 \pm 2.67\%$	0.934 ± 0.242

Sampled versus greedy. For the unweighted mean across cities, greedy execution achieved 3182.6 vehicles/hour, 61.89% completion, and 0.980 s/m wait density, compared with 3345.6, 64.93%, and 0.613 s/m for sampled execution.

Coverage study. The 50%-coverage training study used five training layouts and three held-out traffic seeds on each of five evaluation layouts at 25%, 50%, and 75% coverage; the 100% condition used one layout and the same three seeds. Each paired observational unit is one evaluation layout–traffic-seed combination: $n = 15$ at each partial-coverage condition and $n = 3$ at 100% coverage. For sampled execution, PPO – max-pressure throughput differences at 25%, 50%, 75%, and 100% coverage were +5.7, -28.9 , -104.3 , and -108.0 vehicles/hour; completion differences were +0.24, -0.66 , -2.99 , and -3.14 percentage points. Paired 95% Student- t interval half-widths for throughput were 52.5, 54.1, 41.3, 345.2 vehicles/hour, respectively; the 100% condition consequently has a very wide interval.

C Concrete graph example

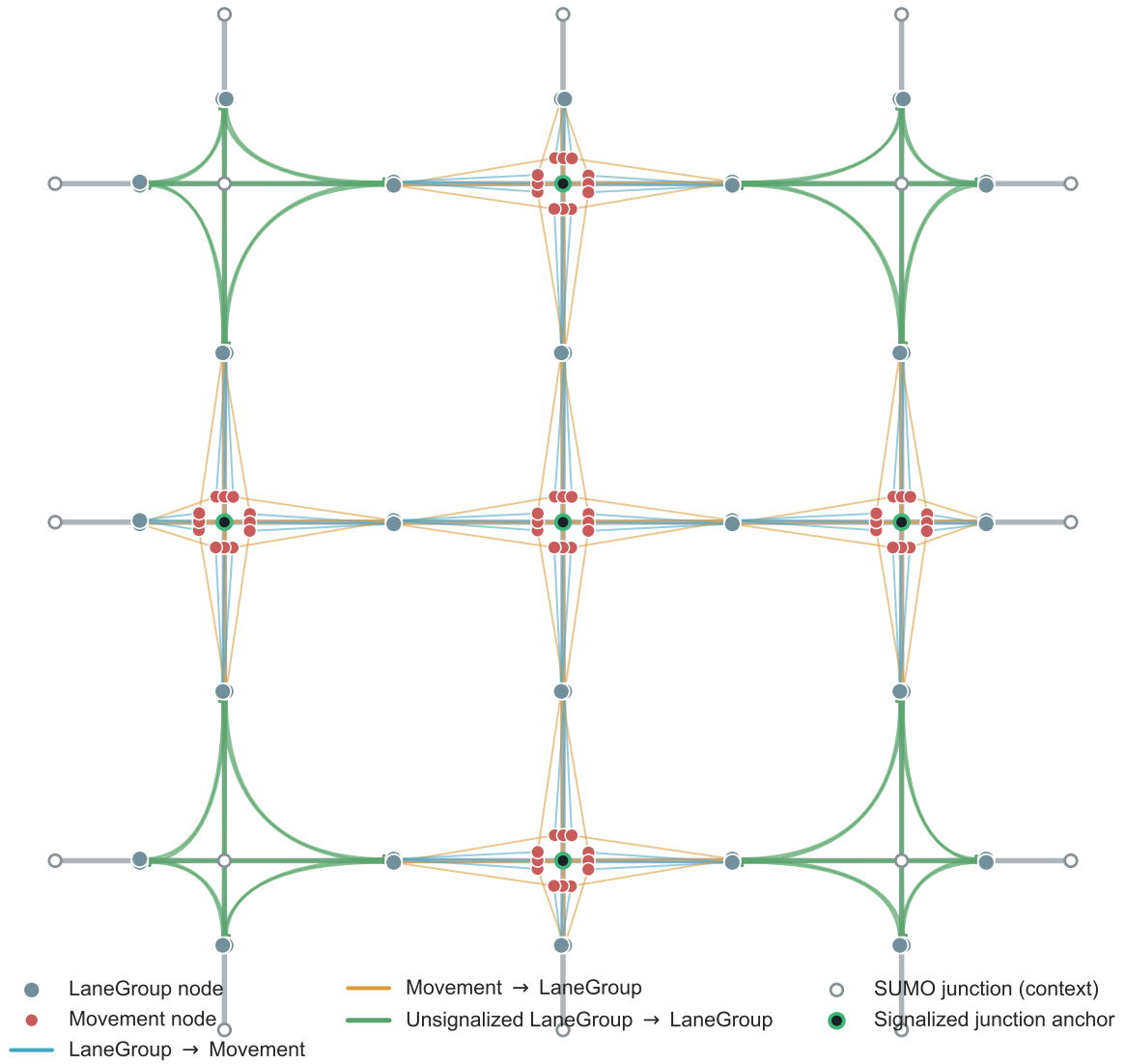


Figure 5: Complete 3×3 synthetic road layout represented as the controller graph. LaneGroup nodes follow directed corridors; Movement nodes encode legal input-to-output paths at signalized junctions. Cyan and amber edges are the typed LaneGroup-to-Movement and Movement-to-LaneGroup relations, while green edges propagate LaneGroup context through unsignalized junctions. Junction markers provide spatial context only and are not GNN nodes. Positions are generated from the same embedded graph data used by the interactive HTML visualization.